Manipulation with ROS2

Unit 4: Cartesian Paths & Kinematics Plugin

- Summary -

Estimated time for completion: **4h**

This unit will show you how to perform Cartesian Paths with MoveIt. Additionally, you will learn how to change the Kinematics plugin for MoveIt.

- End of Summary -

Until now, you've been performing motions in a **"free-space"**. This means, you plan and execute a motion from a point A to a point B, but you don't really care about how it gets there. In general terms, this usually is enough. However, there are some particular cases (for instance, in some industrial environments) where you do need to have control over where the end-effector is pointing at every moment.

For this cases, we will introduce you to Cartesian Path Planning. Cartesian Paths allow you to specify a list of waypoints for the end-effector to go through. Let's see a simple example!

4.1 Cartesian Paths

Let's create a simple example to see how you can create a Cartesian Path.

- Exercise 4.1 -

Let's have a quick look at the code that you will be executing:

cartesian_path.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>

#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  rclcpp::NodeOptions node_options;
  node_options.automatically_declare_parameters_from_overrides(true);
  auto move_group_node =
      rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);
```

```
rclcpp::executors::SingleThreadedExecutor executor;
executor.add_node(move_group_node);
std::thread([&executor]() { executor.spin(); }).detach();

static const std::string PLANNING_GROUP_ARM = "ur_manipulator";

moveit::planning_interface::MoveGroupInterface move_group_arm(
    move_group_node, PLANNING_GROUP_ARM);

const moveit::core::JointModelGroup *joint_model_group_arm =
    move_group_arm.getCurrentState()->getJointModelGroup(PLANNING_GROUP_ARM);

// Get Current State
moveit::core::RobotStatePtr current_state_arm =
    move_group_arm.getCurrentState(10);

std::vector<double> joint_group_positions_arm;
current_state_arm->copyJointGroupPositions(joint_model_group_arm,
                                           joint_group_positions_arm);

move_group_arm.setStartStateToCurrentState();

// Go Home
RCLCPP_INFO(LOGGER, "Going Home");

// joint_group_positions_arm[0] = 0.00;  // Shoulder Pan
joint_group_positions_arm[1] = -2.50; // Shoulder Lift
joint_group_positions_arm[2] = 1.50;  // Elbow
joint_group_positions_arm[3] = -1.50; // Wrist 1
joint_group_positions_arm[4] = -1.55; // Wrist 2
// joint_group_positions_arm[5] = 0.00;  // Wrist 3

move_group_arm.setJointValueTarget(joint_group_positions_arm);

moveit::planning_interface::MoveGroupInterface::Plan my_plan_arm;
bool success_arm = (move_group_arm.plan(my_plan_arm) ==
                    moveit::core::MoveItErrorCode::SUCCESS);

move_group_arm.execute(my_plan_arm);

// Pregrasp
RCLCPP_INFO(LOGGER, "Pregrasp Position");

geometry_msgs::msg::Pose target_pose1;
target_pose1.orientation.x = -1.0;
target_pose1.orientation.y = 0.00;
```

```
target_pose1.orientation.z = 0.00;
target_pose1.orientation.w = 0.00;
target_pose1.position.x = 0.343;
target_pose1.position.y = 0.132;
target_pose1.position.z = 0.264;
move_group_arm.setPoseTarget(target_pose1);

success_arm = (move_group_arm.plan(my_plan_arm) ==
            moveit::core::MoveItErrorCode::SUCCESS);

move_group_arm.execute(my_plan_arm);

// Approach
RCLCPP_INFO(LOGGER, "Approach to object!");

std::vector<geometry_msgs::msg::Pose> approach_waypoints;
target_pose1.position.z -= 0.03;
approach_waypoints.push_back(target_pose1);

target_pose1.position.z -= 0.03;
approach_waypoints.push_back(target_pose1);

moveit_msgs::msg::RobotTrajectory trajectory_approach;
const double jump_threshold = 0.0;
const double eef_step = 0.01;

double fraction = move_group_arm.computeCartesianPath(
   approach_waypoints, eef_step, jump_threshold, trajectory_approach);

move_group_arm.execute(trajectory_approach);

// Retreat

RCLCPP_INFO(LOGGER, "Retreat from object!");

std::vector<geometry_msgs::msg::Pose> retreat_waypoints;
target_pose1.position.z += 0.03;
retreat_waypoints.push_back(target_pose1);

target_pose1.position.z += 0.03;
retreat_waypoints.push_back(target_pose1);

moveit_msgs::msg::RobotTrajectory trajectory_retreat;

fraction = move_group_arm.computeCartesianPath(
   retreat_waypoints, eef_step, jump_threshold, trajectory_retreat);
```

```
  move_group_arm.execute(trajectory_retreat);

  rclcpp::shutdown();
  return 0;
}
```

Most of the code is the same that you used in the previous unit, so let's have a look at the new parts.

First of all, let's have a look at the new `cartesian path` section:

In [ ]:

```
std::vector<geometry_msgs::msg::Pose> approach_waypoints;
target_pose1.position.z -= 0.03;
approach_waypoints.push_back(target_pose1);

target_pose1.position.z -= 0.03;
approach_waypoints.push_back(target_pose1);

moveit_msgs::msg::RobotTrajectory trajectory_approach;
const double jump_threshold = 0.0;
const double eef_step = 0.01;

double fraction = move_group_arm.computeCartesianPath(
  approach_waypoints, eef_step, jump_threshold, trajectory_approach);

move_group_arm.execute(trajectory_approach);
```

We start by declaring an `approach_waypoints` variable, which is a vector of Poses:

In [ ]:

```
std::vector<geometry_msgs::msg::Pose> approach_waypoints;
```

Then, we decreasse the value of `target_pose1.position.z` by 3 cms, and we add this new Pose to the `approach_waypoints` vector.

In [ ]:

```
target_pose1.position.z -= 0.03;
approach_waypoints.push_back(target_pose1);
```

Next, we decrease again the value of `target_pose1.position.z` by another 3 cms, and we add this new Pose to the `approach_waypoints` vector.

In [ ]:

```
target_pose1.position.z -= 0.03;
approach_waypoints.push_back(target_pose1);
```

Next we declare the variables we need for computing the cartesian path:

In [ ]:

```
moveit_msgs::msg::RobotTrajectory trajectory_approach;
const double jump_threshold = 0.0;
const double eef_step = 0.01;
```

Finally, we call the `computeCartesianPath` method in order to compute the cartesian path:

In [ ]:

```
double fraction = move_group_arm.computeCartesianPath(
    approach_waypoints, eef_step, jump_threshold, trajectory_approach);
```

As you can see, we pass the `approach_waypoints` vector to the method. Also, by setting the `eef_step` variable to `0.01` we indicate that the cartesian path will be interpolated at a resolution of 1 cm. By setting `jump_threshold` to 0.0 we are basically disabling it.
Finally, we just execute the computed cartesian path:

In [ ]:

```
move_group_arm.execute(trajectory_approach);
```

For the retreat motion we are doing exactly the same, but increasing the value of `target_pose1.position.z` instead.

Let's create the files and launch them

a) Inside the **moveit2_scripts** package, create a new C++ program named **cartesian_path.cpp** and paste the code show above.
b) Create also a new launch file to launch the program, called **cartesian_path.launch.py**.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts
mkdir launch
touch launch/cartesian_path.launch.py
```

Paste the code below into this launch file:

cartesian_path.launch.py

In [ ]:

```
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import ExecuteProcess
from ament_index_python.packages import get_package_share_directory
from moveit_configs_utils import MoveItConfigsBuilder


def generate_launch_description():
```

```
    moveit_config = MoveItConfigsBuilder("name",
package_name="my_moveit_config").to_moveit_configs()

    # MoveItCpp demo executable
    moveit_cpp_node = Node(
        name="cartesian_path",
        package="moveit2_scripts",
        executable="cartesian_path",
        output="screen",
        parameters=[
            moveit_config.robot_description,
            moveit_config.robot_description_semantic,
            moveit_config.robot_description_kinematics,
            {'use_sim_time': True},
        ],
    )

    return LaunchDescription(
        [moveit_cpp_node]
    )
```

c) Now, update the **CMakeLists.txt** file to compile everything.

Add to CMakeLists.txt

In [ ]:

```
# Generate the executable
add_executable(cartesian_path
    src/cartesian_path.cpp)
target_include_directories(cartesian_path
    PUBLIC include)
ament_target_dependencies(cartesian_path
    ament_cmake
            rclcpp
            rclcpp_action
            moveit_core
            moveit_ros_planning_interface
            interactive_markers
            moveit_ros_planning
            control_msgs)

# Install the executable
install(TARGETS
            cartesian_path
    DESTINATION lib/${PROJECT_NAME}
)
```

d) Great! Let's now compile our package.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws
colcon build --packages-select moveit2_scripts
source install/setup.bash
```

e) Now test your new program! First, you'll need to launch the MoveIt2 RVIZ environment.

- Reminder -

Remember that before being able to use the move group interface, you need to start the move group node and, optionally, moveit rviz for visualization purposes.

Execute in Shell #1

In [ ]:

```
ros2 launch my_moveit_config move_group.launch.py
```

Execute in Shell #2

In [ ]:

```
ros2 launch my_moveit_config moveit_rviz.launch.py
```

- End of Reminder -

f) Now, run your code:

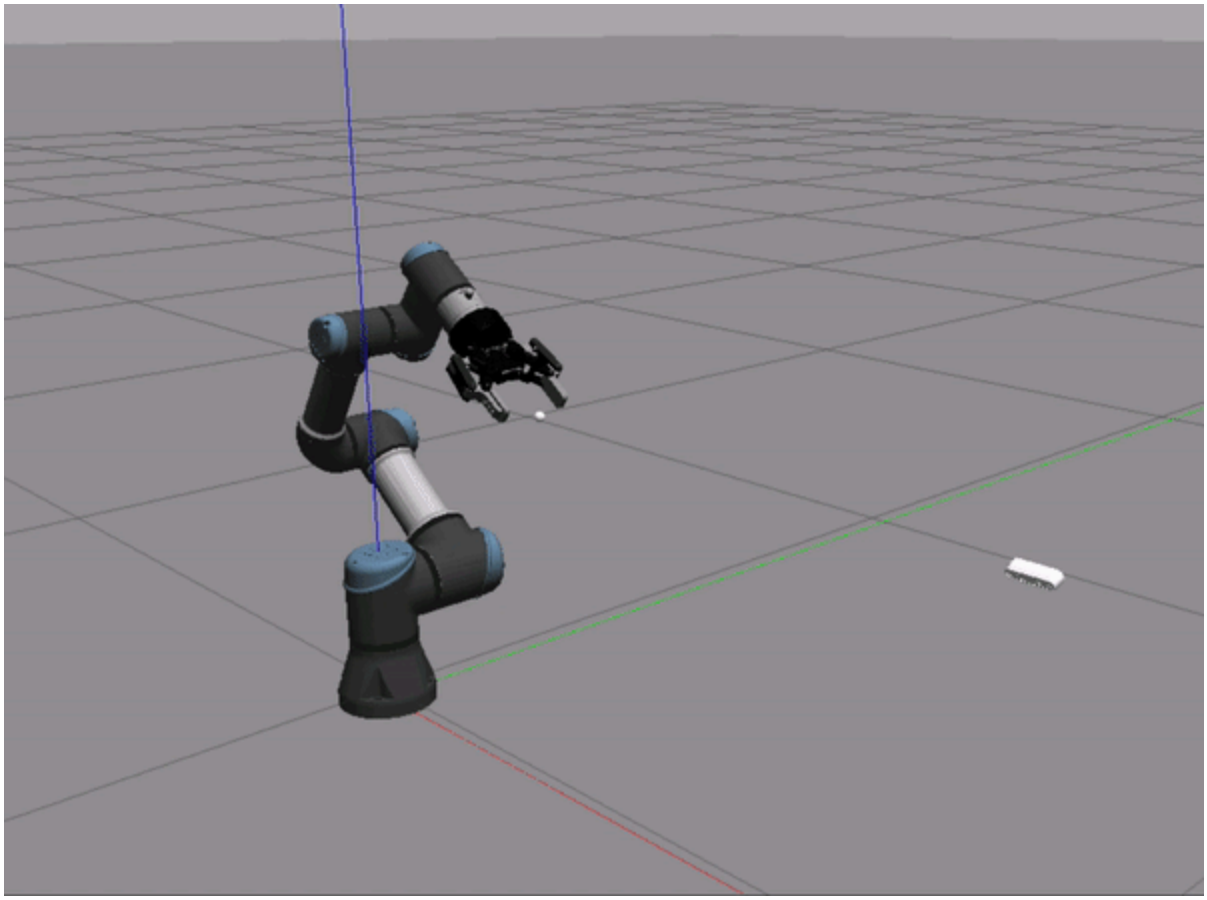
Execute in Shell #4

In [ ]:

```
ros2 launch moveit2_scripts cartesian_path.launch.py
```

- Expected Result for Exercise 4.1 -

You will see the robot going to the **pregrasp** position. Then, it will move the end-effector 6 cm down and 6 cm up alongside the Z axis, following the waypoints provided.



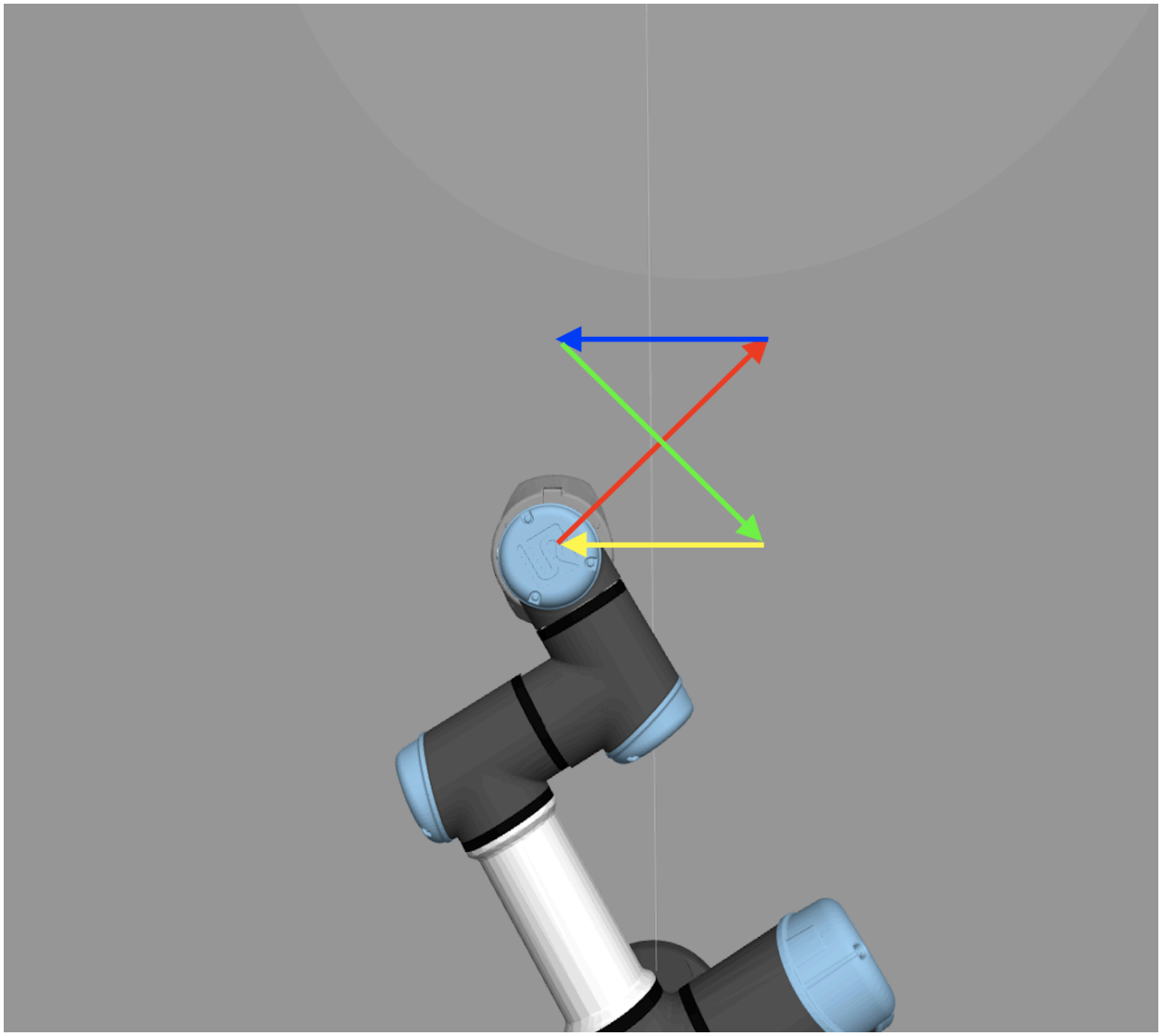
- End of Expected Result -

- End of Exercise 4.1 -

- Exercise 4.2 -

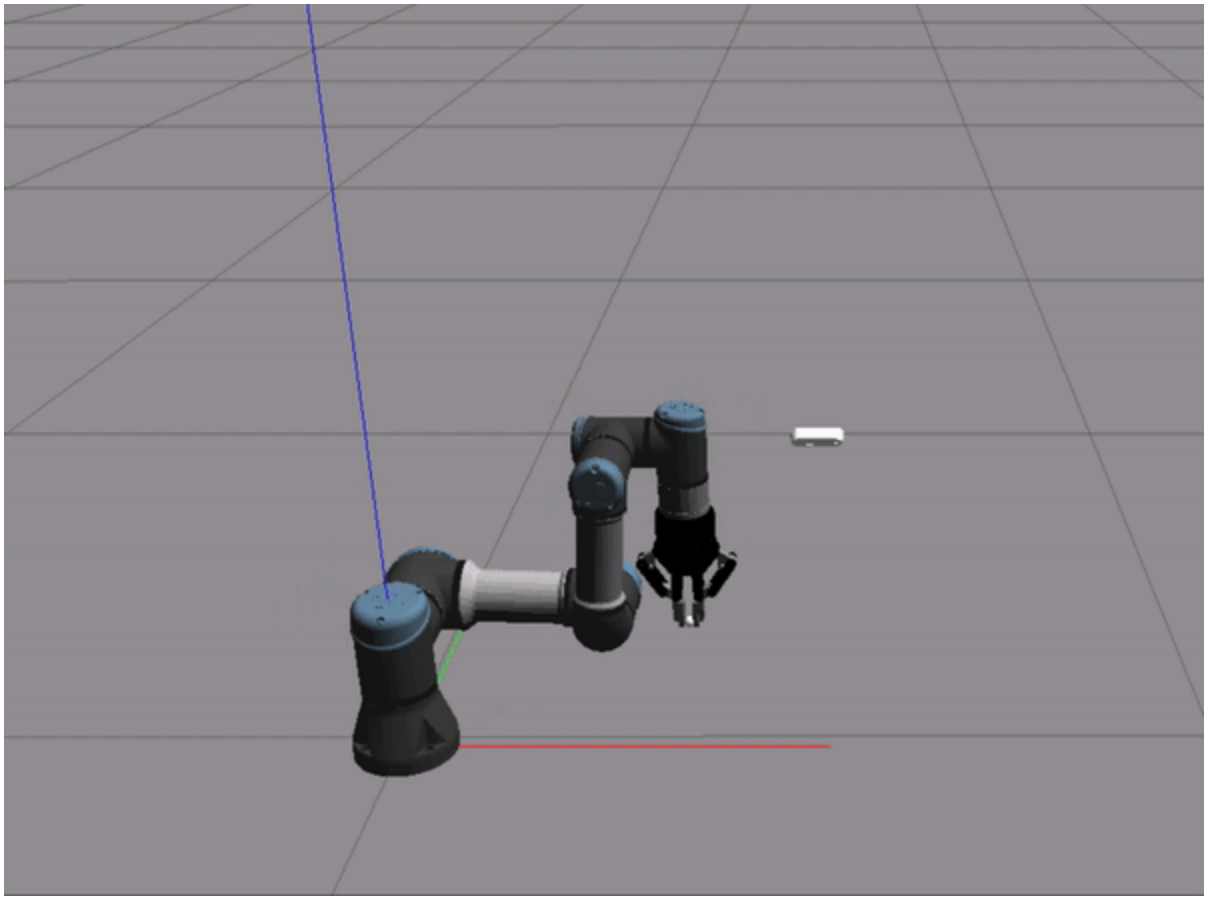
a) Create a new C++ program name **`draw_x.cpp`**. Inside this program, paste the code of `cartesian_path.cpp`.

b) Modify the code in order to "draw" an X in the floor with the robot's end effector.
Expected motion:



- End of Exercise 4.2 -

- Expected Result for Exercise 4.2 -



- End of Expected Result -

- Solution for Exercise 4.2 -

draw_x.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>

#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  rclcpp::NodeOptions node_options;
  node_options.automatically_declare_parameters_from_overrides(true);
  auto move_group_node =
      rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(move_group_node);
  std::thread([&executor]() { executor.spin(); }).detach();
```

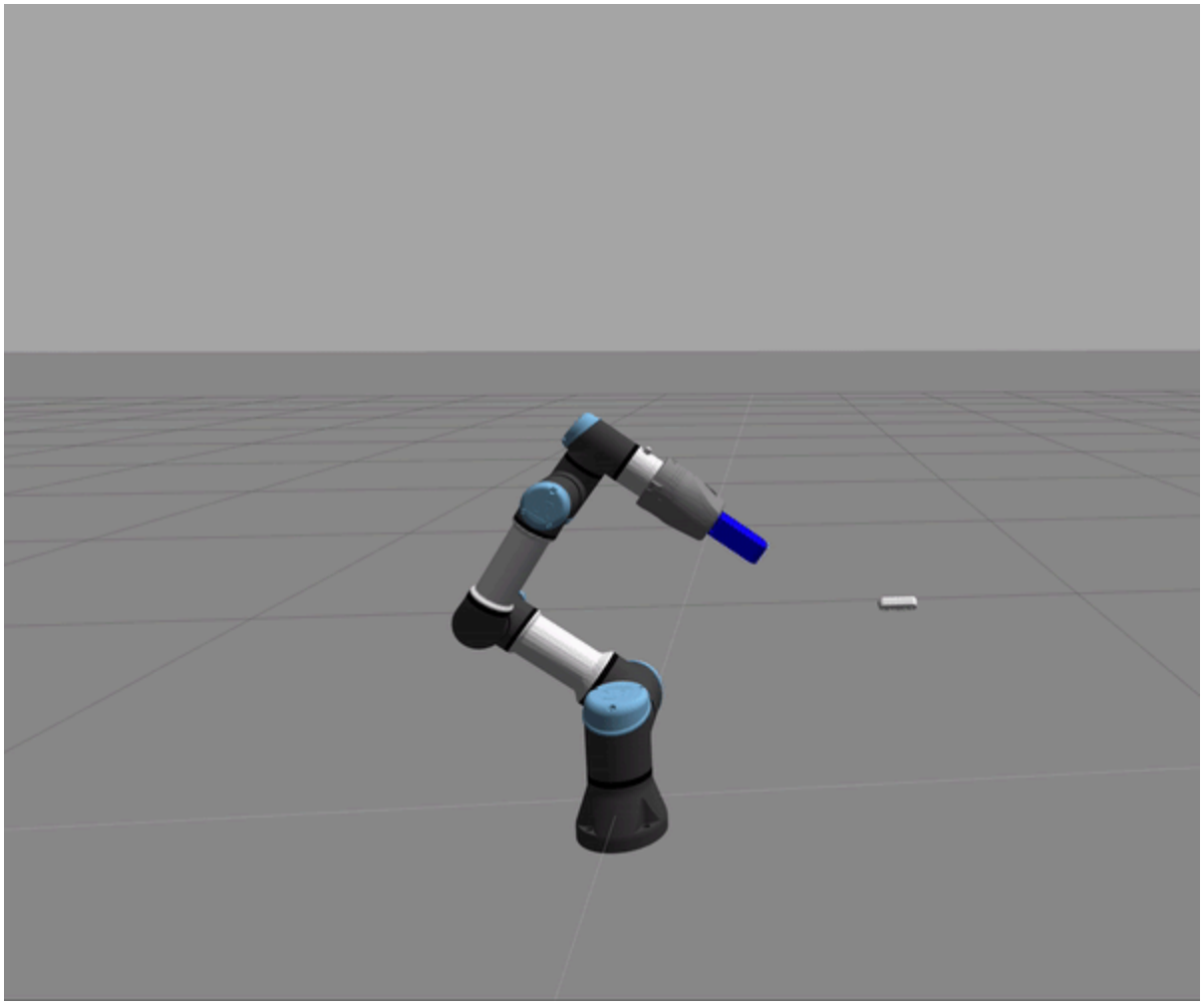
```
static const std::string PLANNING_GROUP_ARM = "ur_manipulator";

moveit::planning_interface::MoveGroupInterface move_group_arm(
    move_group_node, PLANNING_GROUP_ARM);

const moveit::core::JointModelGroup *joint_model_group_arm =
    move_group_arm.getCurrentState()->getJointModelGroup(PLANNING_GROUP_ARM);

// Get Current State
moveit::core::RobotStatePtr current_state_arm =
    move_group_arm.getCurrentState(10);

std::vector<double> joint_group_positions_arm;
current_state_arm->copyJointGroupPositions(joint_model_group_arm,
                                           joint_group_positions_arm);

move_group_arm.setStartStateToCurrentState();

// Pregrasp
RCLCPP_INFO(LOGGER, "Pregrasp Position");

geometry_msgs::msg::Pose target_pose1;
target_pose1.orientation.x = -1.0;
target_pose1.orientation.y = 0.00;
target_pose1.orientation.z = 0.00;
target_pose1.orientation.w = 0.00;
target_pose1.position.x = 0.343;
target_pose1.position.y = 0.132;
target_pose1.position.z = 0.264;
move_group_arm.setPoseTarget(target_pose1);

moveit::planning_interface::MoveGroupInterface::Plan my_plan_arm;
bool success_arm = (move_group_arm.plan(my_plan_arm) ==
                    moveit::core::MoveItErrorCode::SUCCESS);

move_group_arm.execute(my_plan_arm);

// X motion

RCLCPP_INFO(LOGGER, "Draw an X!");

std::vector<geometry_msgs::msg::Pose> waypoints;

target_pose1.position.x += 0.05;
target_pose1.position.y -= 0.05;
```

```
  waypoints.push_back(target_pose1);

  target_pose1.position.y += 0.05;
  waypoints.push_back(target_pose1);

  target_pose1.position.x -= 0.05;
  target_pose1.position.y -= 0.05;
  waypoints.push_back(target_pose1);

  target_pose1.position.y += 0.05;
  waypoints.push_back(target_pose1);

  moveit_msgs::msg::RobotTrajectory trajectory;
  const double jump_threshold = 0.0;
  const double eef_step = 0.01;

  double fraction = move_group_arm.computeCartesianPath(
      waypoints, eef_step, jump_threshold, trajectory);

  move_group_arm.execute(trajectory);

  rclcpp::shutdown();
  return 0;
}
```

- End Solution for Exercise 4.2 -

4.2 Kinematics Plugin

You may have noticed that sometimes, when planning to an end-effector pose, the robot performs unnecessarily complicated trajectories. For instance:



When planning to an end-effector pose, MoveIt2 generates the trajectory by computing the Inverse Kinematics (IK). For this, it uses a **Kinematics Plugin**.
The default inverse kinematics plugin for MoveIt is the `KDLKinematicsPlugin`. This plugin is automatically configured by the MoveIt2 Setup Assistant. You can find it in the `kinematics.yaml` file:

In [ ]:

```
kdl_kinematics_plugin/KDLKinematicsPlugin
```

Of course, MoveIt2 allows you to change this plugin for optimizing the results. In the next exercise we will see the effects of using a different Kinematics Plugin for solving IK.

- Exercise 4.3 -

a) Inside the **moveit2_scripts** package, create a new C++ program named `test_kinematics_plugin.cpp` and paste the below code:

test_kinematics_plugin.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
```

```
#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  rclcpp::NodeOptions node_options;
  node_options.automatically_declare_parameters_from_overrides(true);
  auto move_group_node =
      rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(move_group_node);
  std::thread([&executor]() { executor.spin(); }).detach();

  static const std::string PLANNING_GROUP_ARM = "ur_manipulator";

  moveit::planning_interface::MoveGroupInterface move_group_arm(
      move_group_node, PLANNING_GROUP_ARM);

  const moveit::core::JointModelGroup *joint_model_group_arm =
      move_group_arm.getCurrentState()->getJointModelGroup(PLANNING_GROUP_ARM);

  // Get Current State
  moveit::core::RobotStatePtr current_state_arm =
      move_group_arm.getCurrentState(10);

  std::vector<double> joint_group_positions_arm;
  current_state_arm->copyJointGroupPositions(joint_model_group_arm,
                                             joint_group_positions_arm);

  move_group_arm.setStartStateToCurrentState();

  // Go Home
  RCLCPP_INFO(LOGGER, "Going Home");

  // joint_group_positions_arm[0] = 0.00;  // Shoulder Pan
  joint_group_positions_arm[1] = -2.50; // Shoulder Lift
  joint_group_positions_arm[2] = 1.50;  // Elbow
  joint_group_positions_arm[3] = -1.50; // Wrist 1
  joint_group_positions_arm[4] = -1.55; // Wrist 2
  // joint_group_positions_arm[5] = 0.00;  // Wrist 3

  move_group_arm.setJointValueTarget(joint_group_positions_arm);
```

```
  moveit::planning_interface::MoveGroupInterface::Plan my_plan_arm;
  bool success_arm = (move_group_arm.plan(my_plan_arm) ==
                      moveit::core::MoveItErrorCode::SUCCESS);

  move_group_arm.execute(my_plan_arm);

  // Pregrasp
  RCLCPP_INFO(LOGGER, "Pregrasp Position");

  geometry_msgs::msg::Pose target_pose1;
  target_pose1.orientation.x = -1.0;
  target_pose1.orientation.y = 0.00;
  target_pose1.orientation.z = 0.00;
  target_pose1.orientation.w = 0.00;
  target_pose1.position.x = 0.343;
  target_pose1.position.y = 0.132;
  target_pose1.position.z = 0.264;
  move_group_arm.setPoseTarget(target_pose1);

  success_arm = (move_group_arm.plan(my_plan_arm) ==
                 moveit::core::MoveItErrorCode::SUCCESS);

  move_group_arm.execute(my_plan_arm);

  rclcpp::shutdown();
  return 0;
}
```

The presented program does basically 2 things:

- First, it plans to the home position using FK (no Kinematics plugin involved)
- Second, it plans to the pregrasp pose using IK (Kinematics plugin involved)

b) Create a new launch file named `test_kinematics_plugin.launch.py` for starting the program:

test_kinematics_plugin.launch.py

In [ ]:

```
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import ExecuteProcess
from ament_index_python.packages import get_package_share_directory
from moveit_configs_utils import MoveItConfigsBuilder


def generate_launch_description():
```

```
    moveit_config = MoveItConfigsBuilder("name",
package_name="my_moveit_config").to_moveit_configs()

    # MoveItCpp demo executable
    moveit_cpp_node = Node(
        name="test_kinematics_plugin",
        package="moveit2_scripts",
        executable="test_kinematics_plugin",
        output="screen",
        parameters=[
            moveit_config.robot_description,
            moveit_config.robot_description_semantic,
            moveit_config.robot_description_kinematics,
            {'use_sim_time': True},
        ],
    )

    return LaunchDescription(
        [moveit_cpp_node]
    )
```

c) Don't forget to update the `CMakeLists.txt` file and compile to generate the new executable:

Add to CMakeLists.txt

In [ ]:

```
# Generate the executable
add_executable(test_kinematics_plugin
    src/test_kinematics_plugin.cpp)
target_include_directories(test_kinematics_plugin
    PUBLIC include)
ament_target_dependencies(test_kinematics_plugin
    ament_cmake
            rclcpp
            rclcpp_action
            moveit_core
            moveit_ros_planning_interface
            interactive_markers
            moveit_ros_planning
            control_msgs)

# Install the executable
install(TARGETS
            test_kinematics_plugin
    DESTINATION lib/${PROJECT_NAME}
)
```

Execute in Shell

In [ ]:

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

d) Execute the launch file and analyze the results obtained. Execute it more than 1 time to have multiple results.

Execute in Shell

In [ ]:

```
ros2 launch moveit2_scripts test_kinematics_plugin.launch.py
```

You might get some results where the IK is solved easily, and other results where the resulting trajectory is unnecessarily complex.

e) Let's now change the Kinematics Plugin used by MoveIt2. For this, open the `kinematics.yaml` file and replace its contents with the following:

kinematics.yaml

In [ ]:

```
ur_manipulator:
  #kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
  kinematics_solver: lma_kinematics_plugin/LMAKinematicsPlugin
  kinematics_solver_search_resolution: 0.0050000000000000001
  kinematics_solver_timeout: 0.0050000000000000001
```

As you can see, we have commented out the `KDLKinematicsPlugin` plugin and added the `LMAKinematicsPlugin`.

f) Let's recompile everything to make sure that the new Kinematics Plugin is used:

Execute in Shell

In [ ]:

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

g) You will have to restart Moveit2 for the changes to take effect:

Execute in Shell

In [ ]:

```
ros2 launch test_moveit_config move_group.launch.py
```

h) Finally, run again the launch file and analyze the results obtained with the new Kinematics Plugin.

Execute in Shell

In [ ]:

```
ros2 launch moveit2_scripts test_kinematics_plugin.launch.py
```

It is likely that you won't see significant differences between the plugins' performance, as they are quite similar in the end. However, it's important for you to know that you can create your own custom Kinematics Plugin for your robot, which will greatly impact the inverse kinematics (IK) solutions obtained. We hope to cover this topic in more depth in future updates of this course.

- End of Exercise 4.3 -

4.3 Full Pick & Place pipeline

Now that you have a much deeper knowledge about motion planning and Moveit2, let's build a full Pick & Place application!

- Exercise 4.4 -

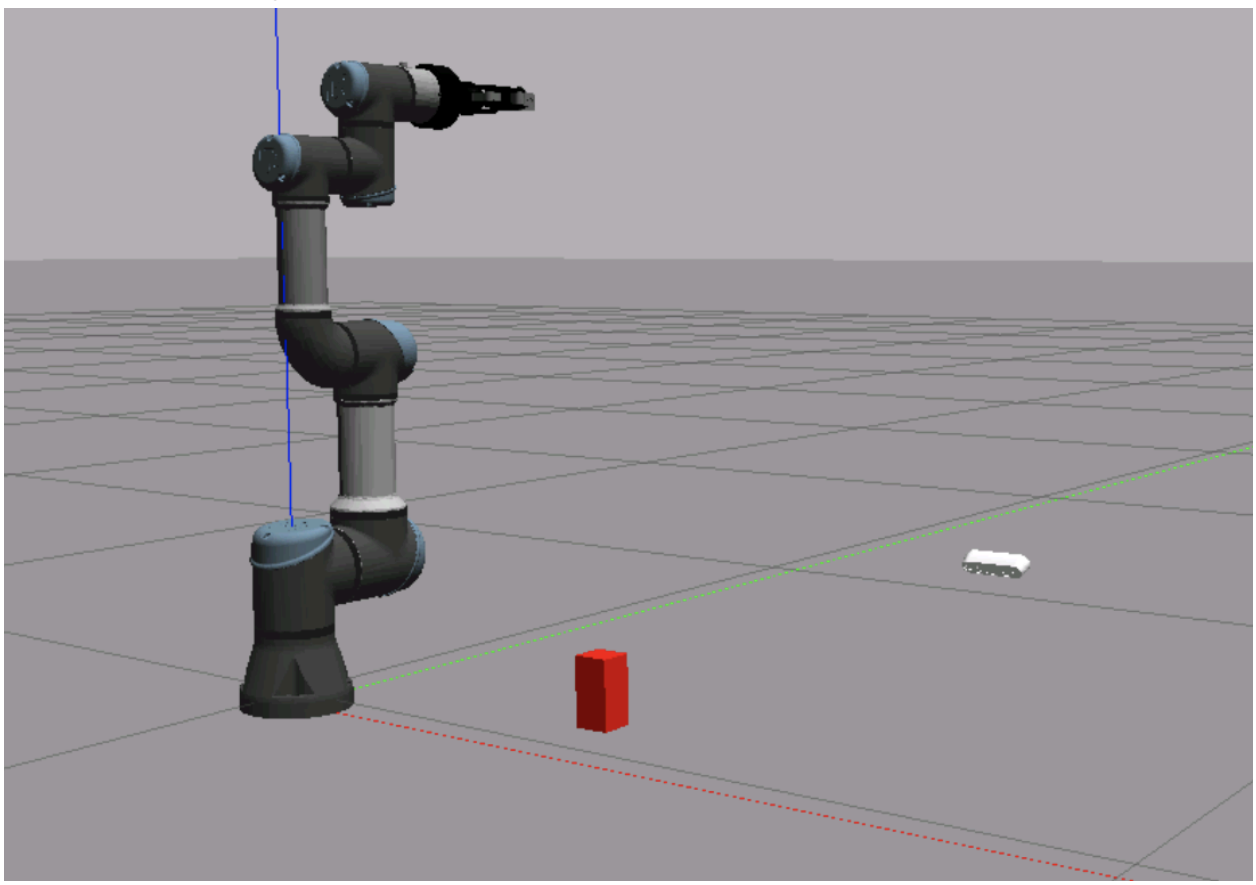
a) Start by spawning the object into the simulation with the following command:

Execute in Shell #2

In [ ]:

```
ros2 run gazebo_ros spawn_entity.py -file ~/ros2_ws/src/grasp_box.urdf -x 0.34 -y 0.13 -z 0.1 -entity grasp_box
```

You will see something like this:



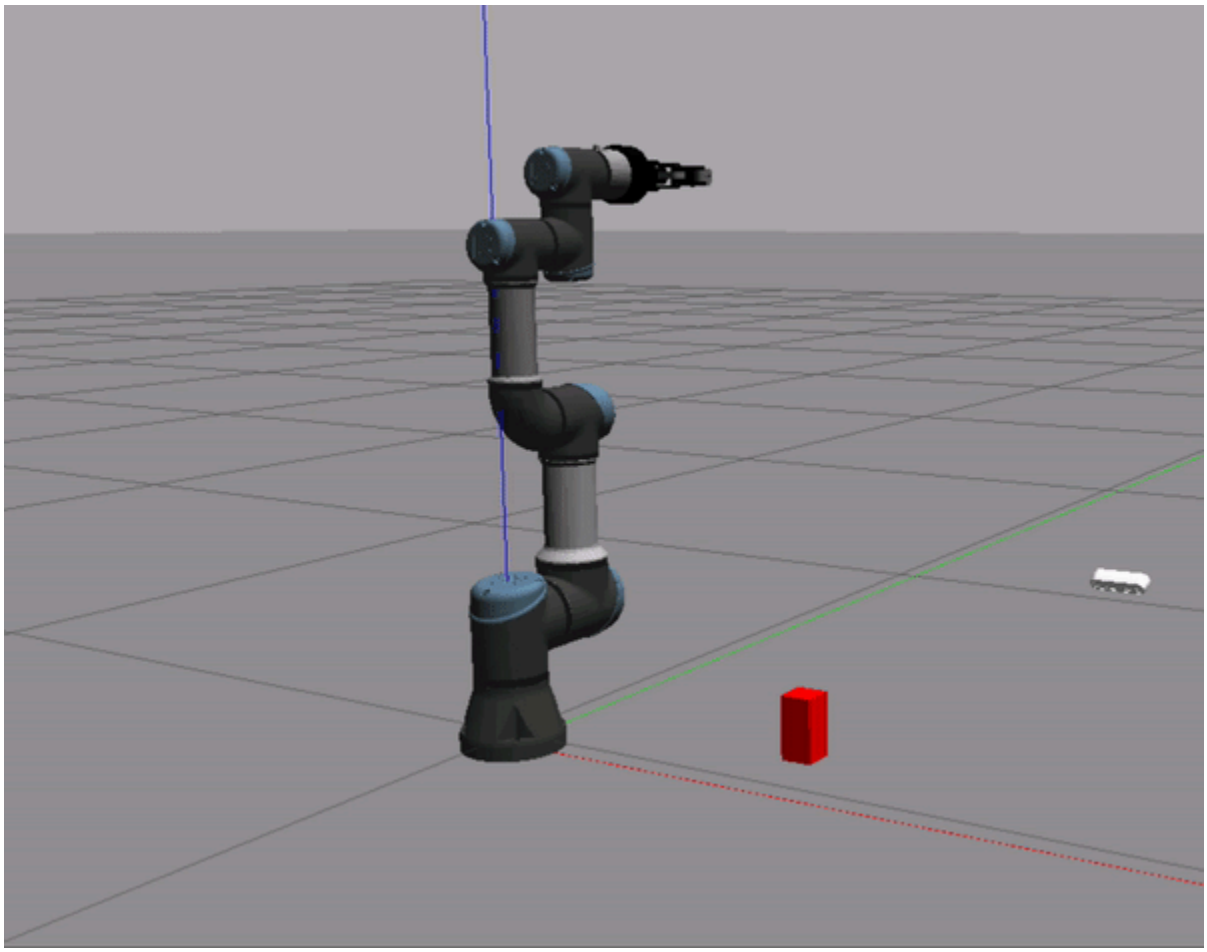
Now, in your **moveit2_scripts** package, create a new file named **pick_place.cpp**.

The program generated should perform the following tasks:

- Go to the **Home** position.
- Go to the **Pick** position.

- Open the gripper.
- Approach the object.
- Close the gripper for picking the object.
- Retreat.
- Go to the Place position.
- Open the gripper to release the object in a new position.

The final result should look something like this:



b) Run the following commands to complete the Pick & Place task:

Execute in Shell 1

In [ ]:

```
ros2 launch my_moveit_connfig move_group.launch.py
```

Execute in Shell 2

In [ ]:

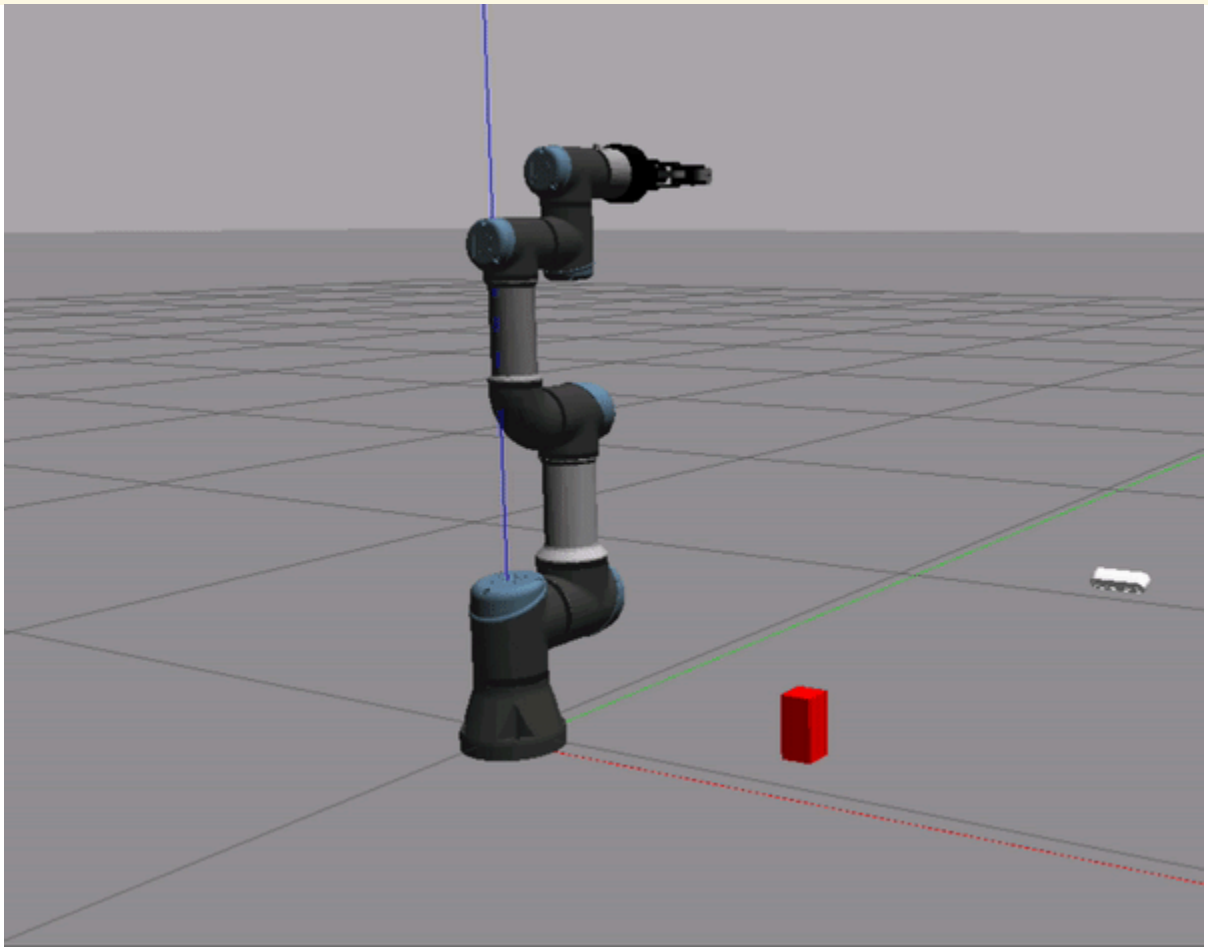
```
ros2 launch my_moveit_connfig moveit_rviz.launch.py
```

Execute in Shell 3

In [ ]:

```
ros2 launch moveit2_scripts pick_place.launch.py
```

- Expected Result for Exercise 4.4 -



- End of Expected Result -

- End of Exercise 4.4 -

- Solution for Exercise 4.4 -

pick_place.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>

#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  rclcpp::NodeOptions node_options;
  node_options.automatically_declare_parameters_from_overrides(true);
  auto move_group_node =
```

```
    rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

rclcpp::executors::SingleThreadedExecutor executor;
executor.add_node(move_group_node);
std::thread([&executor]() { executor.spin(); }).detach();

static const std::string PLANNING_GROUP_ARM = "ur_manipulator";
static const std::string PLANNING_GROUP_GRIPPER = "gripper";

moveit::planning_interface::MoveGroupInterface move_group_arm(
    move_group_node, PLANNING_GROUP_ARM);
moveit::planning_interface::MoveGroupInterface move_group_gripper(
    move_group_node, PLANNING_GROUP_GRIPPER);

const moveit::core::JointModelGroup *joint_model_group_arm =
    move_group_arm.getCurrentState()->getJointModelGroup(PLANNING_GROUP_ARM);
const moveit::core::JointModelGroup *joint_model_group_gripper =
    move_group_gripper.getCurrentState()->getJointModelGroup(
        PLANNING_GROUP_GRIPPER);

// Get Current State
moveit::core::RobotStatePtr current_state_arm =
    move_group_arm.getCurrentState(10);
moveit::core::RobotStatePtr current_state_gripper =
    move_group_gripper.getCurrentState(10);

std::vector<double> joint_group_positions_arm;
std::vector<double> joint_group_positions_gripper;
current_state_arm->copyJointGroupPositions(joint_model_group_arm,
                                           joint_group_positions_arm);
current_state_gripper->copyJointGroupPositions(joint_model_group_gripper,
                                               joint_group_positions_gripper);

move_group_arm.setStartStateToCurrentState();
move_group_gripper.setStartStateToCurrentState();

// Go Home
RCLCPP_INFO(LOGGER, "Going Home");

// joint_group_positions_arm[0] = 0.00;  // Shoulder Pan
joint_group_positions_arm[1] = -2.50; // Shoulder Lift
joint_group_positions_arm[2] = 1.50;  // Elbow
joint_group_positions_arm[3] = -1.50; // Wrist 1
joint_group_positions_arm[4] = -1.55; // Wrist 2
// joint_group_positions_arm[5] = 0.00;  // Wrist 3
```

```
move_group_arm.setJointValueTarget(joint_group_positions_arm);

moveit::planning_interface::MoveGroupInterface::Plan my_plan_arm;
bool success_arm = (move_group_arm.plan(my_plan_arm) ==
                    moveit::core::MoveItErrorCode::SUCCESS);

move_group_arm.execute(my_plan_arm);

// Pregrasp
RCLCPP_INFO(LOGGER, "Pregrasp Position");

geometry_msgs::msg::Pose target_pose1;
target_pose1.orientation.x = -1.0;
target_pose1.orientation.y = 0.00;
target_pose1.orientation.z = 0.00;
target_pose1.orientation.w = 0.00;
target_pose1.position.x = 0.343;
target_pose1.position.y = 0.132;
target_pose1.position.z = 0.264;
move_group_arm.setPoseTarget(target_pose1);

success_arm = (move_group_arm.plan(my_plan_arm) ==
               moveit::core::MoveItErrorCode::SUCCESS);

move_group_arm.execute(my_plan_arm);

// Open Gripper

RCLCPP_INFO(LOGGER, "Open Gripper!");

move_group_gripper.setNamedTarget("open");

moveit::planning_interface::MoveGroupInterface::Plan my_plan_gripper;
bool success_gripper = (move_group_gripper.plan(my_plan_gripper) ==
                        moveit::core::MoveItErrorCode::SUCCESS);

move_group_gripper.execute(my_plan_gripper);

// Approach
RCLCPP_INFO(LOGGER, "Approach to object!");

std::vector<geometry_msgs::msg::Pose> approach_waypoints;
target_pose1.position.z -= 0.03;
approach_waypoints.push_back(target_pose1);

target_pose1.position.z -= 0.03;
```

```
approach_waypoints.push_back(target_pose1);

moveit_msgs::msg::RobotTrajectory trajectory_approach;
const double jump_threshold = 0.0;
const double eef_step = 0.01;

double fraction = move_group_arm.computeCartesianPath(
    approach_waypoints, eef_step, jump_threshold, trajectory_approach);

move_group_arm.execute(trajectory_approach);

// Close Gripper

RCLCPP_INFO(LOGGER, "Close Gripper!");

move_group_gripper.setNamedTarget("close");

success_gripper = (move_group_gripper.plan(my_plan_gripper) ==
                   moveit::core::MoveItErrorCode::SUCCESS);

move_group_gripper.execute(my_plan_gripper);

// Retreat

RCLCPP_INFO(LOGGER, "Retreat from object!");

std::vector<geometry_msgs::msg::Pose> retreat_waypoints;
target_pose1.position.z += 0.03;
retreat_waypoints.push_back(target_pose1);

target_pose1.position.z += 0.03;
retreat_waypoints.push_back(target_pose1);

moveit_msgs::msg::RobotTrajectory trajectory_retreat;

fraction = move_group_arm.computeCartesianPath(
    retreat_waypoints, eef_step, jump_threshold, trajectory_retreat);

move_group_arm.execute(trajectory_retreat);

// Place

RCLCPP_INFO(LOGGER, "Rotating Arm");

current_state_arm = move_group_arm.getCurrentState(10);
current_state_arm->copyJointGroupPositions(joint_model_group_arm,
```

```
                                    joint_group_positions_arm);

  joint_group_positions_arm[0] = 1.57; // Shoulder Pan

  move_group_arm.setJointValueTarget(joint_group_positions_arm);

  success_arm = (move_group_arm.plan(my_plan_arm) ==
                 moveit::core::MoveItErrorCode::SUCCESS);

  move_group_arm.execute(my_plan_arm);

  // Open Gripper

  RCLCPP_INFO(LOGGER, "Release Object!");

  move_group_gripper.setNamedTarget("open");

  success_gripper = (move_group_gripper.plan(my_plan_gripper) ==
                     moveit::core::MoveItErrorCode::SUCCESS);

  move_group_gripper.execute(my_plan_gripper);

  rclcpp::shutdown();
  return 0;
}
```

- End Solution for Exercise 4.4 -

English proofread